

OMES-INS-001 · Revision D

OpenCore MES

Installation procedure

A plant's server, a plant folder, upgrades and suites

Document	OMES-INS-001
Revision	D, issued 2026-10-07
Applies to	OpenCore MES as built on the issue date

Contents

1. 1. Before you start

- 2. 1.1 What you install
- 3. 1.2 Requirements
- 4. 1.3 Which way
- 5. 1.4 Decide first

6. 2. Install on a Linux server

7. 3. A plant folder, a MacBook, a Windows laptop

- 8. 3.1 On a laptop

9. 4. Upgrades

10. 5. Suites

11. 6. Reference

- 12. 6.1 Settings
- 13. 6.2 Files and folders
- 14. 6.3 Console commands
- 15. 6.4 When something refuses
- 16. 6.5 Revision history

Procedures

- 1. IN-10 Prepare the server
- 2. IN-11 Harden the host
- 3. IN-12 Install a release
- 4. IN-13 Keys and settings
- 5. IN-14 The service and HTTPS
- 6. IN-15 The database and the first administrator
- 7. IN-16 Wall the script runner in
- 8. IN-17 Protect the audit trail
- 9. IN-18 Backups and the restore test
- 10. IN-19 Alerts and monitoring
- 11. IN-20 First sign-in, and setting the plant up
- 12. IN-21 Before go-live
- 13. IN-30 Install from npm
- 14. IN-31 Prepare a MacBook
- 15. IN-32 Prepare a Windows laptop
- 16. IN-33 Install and run it on a laptop
- 17. IN-40 Upgrade a server
- 18. IN-41 Upgrade a plant folder (npm)
- 19. IN-42 Going back after a bad upgrade, or restoring a backup
- 20. IN-50 Install, update, remove a suite (a plant folder)
- 21. IN-51 Suites on a server

1. Before you start

This procedure installs OpenCore MES for a plant: the server, its database, its keys, its first administrator, backups, and the checks before go-live; then laptops, upgrades and suites. Each procedure is a table of steps with what you should see; tick **Done** as you go, and keep the signed copy with the installation's handover.

How to read a step.

- The words say what to do. The **dark box** under them holds only commands: one whole command a line, nothing else. Copy the box (its **Copy** button) or type it line by line, in the order shown.
- Replace what is **<in yellow, between angle brackets>** before you run a line (the brackets go too): `<server name>` becomes `mes.plant.example`.
- A **light box titled File** is the content of a file: open the file in an editor (the step's dark box does), paste it there, save. Never paste a file's content into the terminal.
- Where there are two ways (from npm or from the source, a Mac or Windows), each way is a step of its own: do the one that fits, skip the other.

1.1 What you install

PostgreSQL: everything the plant keeps (designs, records, the audit trail, sessions), on the same server, reached over the local socket.

OpenCore MES: one Node process (`app/mes/server.mjs`), run by systemd as the user `opencore` , listening on 127.0.0.1 only. It upgrades its own database at each start.

Caddy: HTTPS in front, with a certificate it fetches and renews by itself, and HSTS. The application writes its own Content-Security-Policy.

Backups: nightly, encrypted, kept on the server and copied off it; a restore tested every month (`ops/backup/`).

1.2 Requirements

	What	Notes
Server	Ubuntu 24.04 LTS, x86-64, root access	Sized for your plant (the load figures measured so far are in <code>app/mes/README.md</code>); disk for the database, its backups (14 daily and 12 monthly kept by default) and the event log.
Node.js	<code>>=24</code> (the package's engines)	<code>ops/demo/provision.sh</code> installs the latest release, checked against its published checksum; pin it with <code>NODE_VERSION</code> .
PostgreSQL	16 or later	The test pipeline runs on 16. Ubuntu 24.04's own package is used.
Network	A DNS name for the server; ports 80 and 443 open to the plant's users	Outbound: your identity provider or directory, the systems your services call (ERP...), Let's Encrypt, the suites registry if you use suites, the AI provider if you use the copilot.

	What	Notes
Browsers	Safari 15.4, Chrome 93, Firefox 92 or later	Firefox 114 or later for the rule advice beside a form (older: the server decides at save).
Your workstation	SSH with a key to the server	Chrome only if you print these documents as PDF.
A laptop (trying it)	A Mac on a macOS Homebrew supports, or Windows 10 (2004 or later) or 11 with WSL 2	Node and PostgreSQL as above, installed per IN-31 or IN-32.

1.3 Which way

	Linux server (part 2)	npm plant folder (part 3)	MacBook or Windows laptop (part 3)
For	Production: a plant's live system	A pilot, a small plant	Trying it, training, a demo, designing offline; never a plant's live system
From	A source archive of the version you install (github.com/opencore-mes/opencore-mes)	The npm package @opencore-mes/server (a plant folder holds its settings)	The npm package, or a source archive
Runs as	systemd, its own user, HTTPS by Caddy, hardened host	opencore-mes start ; production practice is part 2's	From a terminal, on the laptop only (http://localhost:9090)
Backups, audit protection, host hardening	Scripts in ops/ , used as they are	Not in the package: take ops/ from the source archive of the same version	None: a laptop's data is not backed up by OpenCore MES

Caution. Versions. The empty installation (`--empty`), the first administrator (`admin`) and the suite commands come with the release this procedure accompanies. The npm package published as `0.1.0` was built before them: with it, a plant folder or a laptop can try the sample plant (IN-33, way A) only; install from a source archive for the rest, or wait for the next package.

1.4 Decide first

Write these down before you start; the procedures ask for them.

	Decision	Where it goes
1	The server's name (e.g. <code>mes.plant.example</code>)	Caddy, the password links (<code>--url</code>)
2	How people sign in: your identity provider (OpenID Connect), your directory (LDAP or Active Directory), or passwords of OpenCore MES's own with a second factor	<code>OIDC_*</code> , <code>LDAP_*</code> , MFA (6.1)
3	The first administrator: their sign-in id, as your directory or identity provider knows them (lower case letters, digits, <code>_</code> and <code>-</code> : an employee number such as <code>104523</code> works; a dot does not), and their full name	IN-15

	Decision	Where it goes
4	The first department, which governs the organization (Engineering unless you say otherwise)	--department
5	The plant's time zone	PLANT_TZ
6	Where backups are copied off the server (a bucket, another host)	BACKUP_COPY
7	Where alerts go (a chat channel's or on-call service's incoming webhook)	ALERT_WEBHOOK_URL
8	Which hosts services may reach (ERP, the identity provider...)	MES_CONNECTION_HOSTS
9	Who keeps the keys, and where the offline copies are	IN-13
10	The copilot: none, or an AI provider and its key	AI_PROVIDER

Caution. Never copy the public demo's settings (`ops/demo/demo.env` : `DEMO=1` , guests, `SEED_*`) nor its reset timer (`opencore-mes-reset.timer` , which wipes its database every night) to a plant. Take the steps here, not the demo's deploy script as it is.

2. Install on a Linux server

Everything below is done on the server, in a root shell (`sudo -i`), unless a step says otherwise. The installation's settings are one file, `/etc/opencore-mes/opencore-mes.env` . Where a step runs something as the service's user with those settings, it first opens that user's shell (IN-15 shows how), and closes it after.

IN-10 Prepare the server

Purpose	PostgreSQL, Caddy, Node, the system user and its directories, the database role.
Done by	IT
Before you start	• A fresh Ubuntu 24.04 server; its DNS name points at it. • A source archive of the version you install, unpacked on your workstation (it holds <code>ops/</code>).

#	Do this	You should see	Done
1	On your workstation, in the unpacked source archive: copy the provisioning script to the server. <pre>scp ops/demo/provision.sh root@<server name>:/root/</pre>	—	
2	On the server, run it with the Node version you choose (leave <code>NODE_VERSION</code> out for the latest). <pre>NODE_VERSION=<v24.11.0> bash /root/provision.sh</pre>	It installs <code>postgresql</code> <code>caddy</code> <code>curl</code> <code>ca-certificates</code> <code>python3</code> <code>xz-utils</code> <code>libatomic1</code> <code>bubblewrap</code> , Node in <code>/opt/node</code> (its checksum checked) with <code>/usr/local/bin/node</code> , the system user <code>opencore</code> , <code>/srv/opencore-mes/releases</code> and <code>/var/lib/opencore-mes/events</code> , the database role <code>opencore</code> (local socket, peer authentication, may create databases) and the role <code>mes_query</code> the query page reads as. It is safe to run again.	
3	Check what it installed. <pre>node -v sudo -u postgres psql -Atc 'select version()' systemctl is-active postgresql caddy</pre>	Node 24 or later; PostgreSQL 16 or later; active twice.	
4	Make the folder for the settings and keys. <pre>install -d -m 0750 -o root -g opencore /etc/opencore-mes</pre>	—	

provision.sh does not install the service, harden the host or set up backups: those are IN-11, IN-14 and IN-18.

IN-11 Harden the host

Purpose	SSH by key only, a firewall, bans after failed SSH logins, security updates.
Done by	IT
Before you start	- You sign in to the server with an SSH key (keep a second session open while you do step 3). - The variables below are named for the demo; the script is for any such server.

#	Do this	You should see	Done
1	On your workstation, in the source archive: see where the server stands. Nothing is changed. <pre>export DEMO_HOST=<server name> DEMO_PORT=<SSH port> DEMO_KEY=<path to your SSH key> ops/demo/harden.sh status</pre>	Listening ports, the firewall, SSH's settings, updates, fail2ban, backup timers, time sync.	
2	Still on your workstation (same terminal): SSH by key only. <pre>ops/demo/harden.sh ssh</pre> Cannot be undone. Passwords stop working for SSH at once. If your key does not sign in, you are locked out of SSH and need the host's console to get back in: keep the second session open until you have signed in again with the key.	/etc/ssh/sshd_config.d/00-0-opencore-mes.conf : no passwords, root by key only, 3 tries; it checks that your key still signs in.	
3	The firewall. <pre>ops/demo/harden.sh firewall</pre> Cannot be undone. Everything not let through is refused at once. If the SSH port you gave is not the one you use, you are locked out and need the host's console.	ufw: incoming denied but your SSH port, 80 and 443 (TCP, and UDP for HTTP/3).	
4	Bans after failed SSH logins. <pre>ops/demo/harden.sh fail2ban</pre>	SSH banned for an hour after 5 failures.	
5	On the server: Ubuntu's automatic security updates (not done by the script). Answer Yes when asked. <pre>apt-get install -y unattended-upgrades dpkg-reconfigure -plow unattended-upgrades</pre>	Security updates install by themselves; harden.sh status reports them.	
6	Encrypt the disk or volume that holds /var/lib/postgresql and /var/backups : your host's encrypted volumes, or LUKS when the server is installed.	Data at rest is encrypted. The product does not do this for you.	

IN-12 Install a release

Purpose	The application's files, in a release folder that the service runs from.
Done by	IT
Before you start	The source archive copied to the server (e.g. <code>/root/opencore-mes-<version>.tar.gz</code>). Name the release folder yourself: its version and date, e.g. <code>1.0.0-20261006</code>.

#	Do this	You should see	Done
1	Unpack it as a release folder, owned by the service's user. <pre>mkdir /srv/opencore-mes/releases/<build> tar -xzf /root/<archive>.tar.gz -C /srv/opencore-mes/releases/<build> --strip-components=1 chown -R opencore:opencore /srv/opencore-mes/releases/<build></pre>	The folder holds <code>app/</code> , <code>ops/</code> , <code>src/</code> , <code>package.json</code> .	
2	Install its dependencies. <pre>cd /srv/opencore-mes/releases/<build> sudo -u opencore npm ci --omit=dev --no-audit --no-fund</pre>	<code>node_modules/</code> : the three runtime dependencies.	
3	Let the release name itself (in <code>/healthz</code> and the event log). <pre>echo 'BUILD=<build>' > /srv/opencore-mes/releases/<build>/BUILD.env</pre>	—	
4	Make it the release that runs. <pre>ln -sf /srv/opencore-mes/releases/<build> /srv/opencore-mes/current</pre>	<code>/srv/opencore-mes/current</code> points at it.	

IN-13 Keys and settings

Purpose	The installation's settings, and the two keys kept outside the database and its backups.
Done by	IT, and whoever keeps the keys

#	Do this	You should see	Done
1	Make the two keys: <code>seal.key</code> seals the second-factor secrets (AES-256-GCM); <code>integrity.key</code> seals every record for the data integrity review. <pre>openssl rand -hex 32 > /etc/opencore-mes/seal.key openssl rand -hex 32 > /etc/opencore-mes/integrity.key chown root:opencore /etc/opencore-mes/seal.key /etc/opencore-mes/integrity.key chmod 0440 /etc/opencore-mes/seal.key /etc/opencore-mes/integrity.key</pre> Cannot be undone. Only on a new installation. Run again, the first two lines replace the keys: second factors sealed with the old seal key can no longer be read (everyone sets theirs up again), and every integrity seal reads as changed until a reviewer seals everything again.	64 hex characters each, readable by root and the service only.	
2	Copy both keys off the server, to the place decided (item 9 of 1.4): a password manager, or a sealed envelope in a safe. Never with the database backups. Show each one to copy it: <pre>cat /etc/opencore-mes/seal.key cat /etc/opencore-mes/integrity.key</pre>	Lost, a sealed second factor cannot be read (people set theirs up again), and the integrity seals must be made again by a reviewer.	
3	Open the settings file in an editor. <pre>nano /etc/opencore-mes/opencore-mes.env</pre>	An empty file opens.	

#	Do this	You should see	Done
4	Paste the content below, change it to your decisions, save (Ctrl+O, Enter) and leave (Ctrl+X). Lines starting with # are notes, and stay as they are. File <code>/etc/opencore-mes/opencore-mes.env</code>: paste into an editor, not the terminal <pre># OpenCore MES: this installation's settings. Never in the repository. PROD=1 HOST=127.0.0.1 PORT=3000 DATABASE_URL=postgres:///opencore_mes PGHOST=/var/run/postgresql PLANT_TZ=<Europe/Paris> EVENT_LOG_DIR=/var/lib/opencore-mes/events TRUST_PROXY=1 SEAL_KEY_FILE=/etc/opencore-mes/seal.key INTEGRITY_KEY_FILE=/etc/opencore-mes/integrity.key # How people sign in: one or more of these # (passwords of its own are on unless PASSWORDS=0). # OIDC_ISSUER=https://login.plant.example # OIDC_CLIENT_ID=opencore-mes # OIDC_CLIENT_SECRET=<secret> # LDAP_URL=ldaps://dc.plant.example:636 # LDAP_USER_DN={user}@plant.local MFA=optional # Scripts walled in (IN-16), and the hosts # services may reach. SCRIPT_RUNNER_WRAP=/srv/opencore- mes/current/ops/script-runner-sandbox.sh # SCRIPT_ISOLATION=required MES_CONNECTION_HOSTS= <erp.plant.example,login.plant.example> # Alerts (IN-19). # ALERT_WEBHOOK_URL=<https://hooks.example/...> # The copilot: none unless set. # AI_PROVIDER=anthropic # ANTHROPIC_API_KEY=<key></pre>	—	
5	Let root and the service read it, nobody else. <pre>chown root:opencore /etc/opencore-mes/opencore- mes.env chmod 0640 /etc/opencore-mes/opencore-mes.env</pre>	—	

IN-14 The service and HTTPS

Purpose	OpenCore MES started by systemd, behind Caddy.		
Done by	IT		
#	Do this	You should see	Done
1	Open the service's unit file in an editor. <pre>nano /etc/systemd/system/opencore-mes.service</pre>	—	

#	Do this	You should see	Done
2	Paste the content below (the demo's unit, with the plant's settings file), save and leave. File /etc/systemd/system/opencore-mes.service: paste into an editor, not the terminal <pre>[Unit] Description=OpenCore MES After=network-online.target postgresql.service Wants=network-online.target Requires=postgresql.service [Service] User=opencore Group=opencore WorkingDirectory=/srv/opencore-mes/current EnvironmentFile=/etc/opencore-mes/opencore-mes.env EnvironmentFile=-/srv/opencore-mes/current/BUILD.env ExecStart=/usr/local/bin/node app/mes/server.mjs Restart=always RestartSec=3 NoNewPrivileges=true ProtectSystem=strict ProtectHome=true PrivateTmp=true ReadWritePaths=/var/lib/opencore-mes [Install] WantedBy=multi-user.target</pre>	—	
3	Open Caddy's configuration in an editor; delete what is in it. <pre>nano /etc/caddy/Caddyfile</pre>	—	
4	Paste the content below with your server's name, save and leave. File /etc/caddy/Caddyfile: paste into an editor, not the terminal <pre><mes.plant.example> { encode gzip reverse_proxy 127.0.0.1:3000 header Strict-Transport-Security "max-age=31536000" }</pre>	—	
5	Check it and load it. <pre>caddy validate --config /etc/caddy/Caddyfile systemctl reload caddy</pre>	Valid configuration. Caddy fetches the certificate by itself, and renews it.	
6	Do not start the service yet: the database comes first (IN-15).	—	

IN-15 The database and the first administrator

Purpose	An empty database, and the one person who sets up the rest.
---------	---

Done by	IT
---------	----

#	Do this	You should see	Done
1	Open a shell as the service's user. <pre>sudo -u opencore bash</pre>	The prompt changes to opencore@... .	
2	In that shell: load the settings and go to the release. <pre>set -a; . /etc/opencore-mes/opencore-mes.env; set +a cd /srv/opencore-mes/current</pre>	—	
3	Make the empty database. <pre>node app/mes/db/reset.mjs --empty</pre> Cannot be undone. It makes the database again: everything in it is lost, with no backup made first. Only on a new installation; it refuses a database whose audit trail is protected.	database opencore_mes: empty (the platform's built-ins only, nobody in it) . Nobody can sign in yet.	
4	Name the first administrator: their sign-in id, their full name in quotes, and the server's address. <pre>node app/mes/db/admin.mjs <id> "<Full name>" --url https://<server name></pre> Cannot be undone. It runs once per installation: refused once anybody is in People & departments. Check the sign-in id and the name before you press Enter; afterwards they are changed only through a change in People & departments.	<Name> (<id>) is the first administrator: designer and reviewer, sign-in administrator, approver for Engineering, which governs the organization. Setup is open: ... and a one-time link to set their password, for 3 days (PASSWORD_LINK_DAYS , or --days up to 14).	
5	Only if they sign in through your identity provider or directory, instead of the line above: the same without a password link (their id must be theirs there). <pre>node app/mes/db/admin.mjs <id> "<Full name>" --no-password</pre> Cannot be undone. The same: once per installation. Check the sign-in id and the name first.	The same, without a link.	
6	Leave the service user's shell. <pre>exit</pre>	Back to root's prompt.	
7	Give the link to that person only.	Whoever opens it first sets the password.	

#	Do this	You should see	Done
8	Start the service, and make it start with the server. <pre>systemctl daemon-reload systemctl enable --now opencore-mes systemctl is-active opencore-mes</pre>	active . The journal says what migrated and where it listens.	
9	Ask it how it is. <pre>curl -fsS http://127.0.0.1:3000/healthz</pre>	"ok":true, the build, db up, audit ok, scripts .	
10	Open https://<server name>/ in a browser.	The sign-in page, over HTTPS.	

Caution. Never run `reset.mjs` on a database in use: it makes the database again, and everything in it is lost. It refuses a database whose audit trail is protected (IN-17).

IN-16 Wall the script runner in

Purpose	Rule and service scripts run with no network and no files of the host's: the operating system's walls, not only Node's.
Done by	IT

#	Do this	You should see	Done
1	Install the AppArmor profile that lets bubblewrap make its walls (Ubuntu 23.10 and later). <pre>install -m 0644 /srv/opencore-mes/current/ops/script-runner-sandbox.apparmor /etc/apparmor.d/opencore-mes-bwrap apparmor_parser -r /etc/apparmor.d/opencore-mes-bwrap</pre>	—	
2	Restart, and ask it how its scripts run. <pre>systemctl restart opencore-mes curl -fsS http://127.0.0.1:3000/healthz</pre>	"scripts": { "network": "none", "sandbox": "bwrap", ... }	
3	Open the settings file, and remove the # before <code>SCRIPT_ISOLATION=required</code> ; save and leave. <pre>nano /etc/opencore-mes/opencore-mes.env</pre>	—	
4	Restart. <pre>systemctl restart opencore-mes</pre>	From now on the service refuses to start without its walls, rather than run scripts outside them.	

IN-17 Protect the audit trail

Purpose	The audit trail out of the application's own reach: it may add to it and read it, never change or remove it.
Done by	A database administrator

#	Do this	You should see	Done
1	Run the protection script as the database's owner. <pre>sudo -u postgres psql -v app=opencore -d opencore_mes -f /srv/opencore- mes/current/ops/db/protect-audit.sql</pre> Cannot be undone. On purpose: from then on the application can never change or remove the audit trail, and reset.mjs refuses this database. Run it on the live database only, once the installation is in use.	The trail moves to the schema <code>audit</code> , owned by <code>mes_audit</code> ; the application keeps <code>SELECT</code> , <code>INSERT</code> .	
2	Restart, and check the trail. <pre>systemctl restart opencore-mes curl -fsS http://127.0.0.1:3000/healthz</pre>	<code>"audit":{"ok":true,...}</code>	

Run it again after an upgrade whose migration adds a column to the trail; such a migration fails at start on a protected database until a database administrator runs it as postgres (ops/db/README.md).

IN-18 Backups and the restore test

Purpose	Nightly encrypted backups, copied off the server, and a monthly restore that proves them.
Done by	IT

#	Do this	You should see	Done
1	Make the backup key, readable by root only; copy it off the server with the other keys. <pre>openssl rand -base64 48 > /etc/opencore- mes/backup.key chown root:root /etc/opencore-mes/backup.key chmod 0400 /etc/opencore-mes/backup.key cat /etc/opencore-mes/backup.key</pre> Cannot be undone. Only once. Run again, the first line replaces the key, and every backup made with the old one can no longer be read: keep the old key as long as its backups.	Without it, a backup cannot be read.	
2	Open the backup settings in an editor. <pre>nano /etc/opencore-mes/backup.env</pre>	—	

#	Do this	You should see	Done
3	Paste the content below, with where backups are copied (any command; {} stands for the file), save and leave. File <code>/etc/opencore-mes/backup.env</code>: paste into an editor, not the terminal <pre>BACKUP_DATABASES="postgres:///opencore_mes" BACKUP_DIR=/var/backups/opencore-mes BACKUP_COPY="rclone copy {} <remote>:<path>/"</pre>	—	
4	<pre>chmod 0640 /etc/opencore-mes/backup.env install -d -m 0700 -o postgres -g postgres /var/backups/opencore-mes install -m 644 /srv/opencore- mes/current/ops/backup/opencore-mes-backup.service /srv/opencore-mes/current/ops/backup/opencore-mes- backup.timer /etc/systemd/system/ install -m 644 /srv/opencore- mes/current/ops/backup/opencore-mes-restore- test.service /srv/opencore- mes/current/ops/backup/opencore-mes-restore- test.timer /etc/systemd/system/ systemctl daemon-reload systemctl enable --now opencore-mes-backup.timer opencore-mes-restore-test.timer</pre>	Backups nightly at 02:30 UTC; the restore test on the 1st of each month at 04:00 UTC.	
5	<pre>systemctl start opencore-mes-backup.service ls -lh /var/backups/opencore-mes</pre>	opencore_mes- <stamp>.dump.enc and its .sha256 ; copied where BACKUP_COPY says.	
6	<pre>systemctl start opencore-mes-restore-test.service journalctl -u opencore-mes-restore-test --since -5min -o cat</pre>	restore test: the backup restored into a scratch database, its audit chain verified, then dropped. Keep this output: it is the evidence an auditor asks for.	

Caution. These are nightly backups: a failure loses up to a day. Continuous WAL archiving (pgBackRest or WAL-G) is not set up by these scripts; add it if the plant needs less. Restoring for real: IN-42.

harden.sh backups , from your workstation, does steps 1 to 6 in one go (it backs up every database on the server but test and scratch ones).

IN-19 Alerts and monitoring

Purpose	Hear about it when something breaks.
Done by	IT

#	Do this	You should see	Done
1	Open the settings file; remove the <code>#</code> before <code>ALERT_WEBHOOK_URL</code> and give your webhook's address; save and leave. (<code>ALERT_MIN_SEVERITY</code> is <code>error</code> unless set.) <pre>nano /etc/opencore-mes/opencore-mes.env</pre>	—	
2	Restart. <pre>systemctl restart opencore-mes</pre>	Each event at or above it is posted as <code>{ text, event }</code> , the same kind at most once in ten minutes: the database down, the audit chain broken, data changed outside the platform, a crash.	
3	Set an uptime check from outside, every minute, on <code>https://<server name>/healthz</code> , alerting on anything but 200, and when <code>scripts.network</code> is not <code>none</code> .	—	
4	Ship the journal off the server (your log platform), and watch for failed units (a failed backup fails its unit). <pre>systemctl --failed</pre>	Nothing listed.	

IN-20 First sign-in, and setting the plant up

Purpose	The plant's people, departments and roles, then the end of setup.
Done by	The first administrator

#	Do this	You should see	Done
1	Open the link from IN-15 and set a password (12 characters or more), or sign in through your identity provider.	The designer, with a banner: Setup is open.	
2	Design → People & departments: start a change; add the people, departments and their approval steps, and the roles. Submit it with Execute now (setup) , signing it.	It executes at once, marked setup . Every such change is in the audit trail with your signature.	
3	Optional: what the sign-in page calls the id (a Windows user name, an employee number), the hint in its box (<code>PLANT\username</code>), and the plant's domains a typed id may carry: People & departments → Sign-in , in the same change or another, executed the same way.	The sign-in page shows that label and hint; <code>PLANT\jdoe</code> or <code>jdoe@plant.local</code> signs in as <code>jdoe</code> .	
4	Everyone's first password, for people who sign in with a password here: Design → Sign-in administration → Setup codes for people with no password yet . Choose a department (or everyone) and how many days the codes last, then Make codes and download .	A CSV file: each person's id, name, departments, a code of five capital letters, until when it works, and the address to open. Print the slips (or a mail merge), hand each to its person, then delete the file: it holds the codes.	

#	Do this	You should see	Done
5	Each person opens the sign-in page, clicks I have a setup code , and types their sign-in id, the code and a new password twice.	Their password is set; the code works once. Making codes again for the same people stops the earlier ones.	
6	Only where the plant also signs people in through its directory (Active Directory, LDAP): someone with a password here who has an account there moves to it from Your password (their name, top right): their directory password, then Switch to the directory.	Their password here is taken off; they sign in with the directory's from then on.	
7	Choose the plant's approval level on the same tab: Full (a reviewer, then each department: a regulated plant keeps it), One approval (one approver of a department it touches signs, and it executes) or None (each designer signs their own changes, for a plant with nobody else to ask). Then, when the level's people are there (Full: someone besides the designers who can review; One: an approver of governance besides each designer), clear Setup is open in the same change; execute it the same way. Cannot be undone. Not on one signature any more: opening setup again later is a change that governance reviews and approves.	From then on every change is reviewed and approved. The change says what is still missing if it cannot end yet.	
8	Note in the handover that the installation was set up this way, and that the changes made in setup (Designer → Changes → Setup) are covered by the plant's validation.	—	

The user guide (USERGUIDES.md) is for the people who design the plant from here.

IN-21 Before go-live

Purpose	The installation as it should be, checked and signed.		
Done by	IT and the plant's quality or validation lead		

#	Do this	You should see	Done
1	Ask the service how it is. <pre>curl -fsS http://127.0.0.1:3000/healthz</pre>	ok true; db up; audit.ok true; scripts.network none ; the build you installed.	
2	Run the test pipeline on the server, against a test database (never the live one), as the service's user. <pre>cd /srv/opencore-mes/current sudo -u opencore env PGHOST=/var/run/postgresql TEST_DATABASE_URL=postgres:///opencore_mes_test EVENT_LOG_DIR=/var/lib/opencore-mes/test-events npm run test:all</pre>	all ... stages passed . It makes and drops its own test databases; the name it is given must contain "test", so the live database is safe.	

#	Do this	You should see	Done
3	From your workstation (the variables of IN-11 set): where the server stands. <pre>ops/demo/harden.sh status</pre>	Only your SSH port, 80 and 443 listen from outside; SSH by key; updates on; fail2ban; backup and restore-test timers active; time in sync.	
4	The keys (seal, integrity, backup) are copied off the server, and the backup key is not stored with the backups.	—	
5	The latest restore test passed (IN-18 step 6).	—	
6	Setup has ended (IN-20 step 3), unless the plant decided otherwise and wrote why.	Design → People & departments → Approvals: Setup is open not ticked.	
7	None of the demo's settings or timers are present. <pre>grep -E 'DEMO SEED_' /etc/opencore-mes/opencore-mes.env systemctl list-timers grep opencore-mes</pre>	No DEMO or SEED_ line; only the backup and restore-test timers.	
8	Sign-in policy as decided: MFA , password expiry (PASSWORD_MAX_DAYS), idle sign-out (SESSION_IDLE_MINUTES).	—	

3. A plant folder, a MacBook, a Windows laptop

The npm package is the application, ready to run; a **plant folder** holds an installation's settings (`.env`), its suites and its event log, outside the package. Use it for a pilot or a small plant. For production, run it as part 2 does (systemd, its own user, Caddy, backups, hardening).

IN-30 Install from npm

Purpose	OpenCore MES from its npm package, in a plant folder.
Done by	IT
Before you start	Node 24 or later; PostgreSQL 16 or later, with a role that may create databases, reached over the local socket or <code>DATABASE_URL</code> .

#	Do this	You should see	Done
1	On a Mac, only if Node came from the nodejs.org installer (Homebrew's Node and nvm need nothing): give npm a folder of your own, so the install below does not fail with <code>EACCES</code> . Once per Mac. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.zshrc && source ~/.zshrc</pre>	<code>npm config get prefix</code> then says your <code>~/.npm-global</code> .	
2	On Linux or Windows (Ubuntu), only if Node came from Ubuntu's packages (nvm needs nothing): the same, for bash. Once per machine. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.bashrc && source ~/.bashrc</pre>	The same.	
3	Install the package. <pre>npm install -g @opencore-mes/server opencore-mes version</pre>	<code>opencore-mes version</code> answers.	
4	Make a plant folder, and go into it. <pre>opencore-mes init my-plant cd my-plant</pre>	<code>.env</code> (mode 0600: the settings, commented), <code>suites/</code> , <code>.local/</code> .	
5	Open <code>.env</code> in an editor and set <code>DATABASE_URL</code> (<code>postgres:///opencore_mes</code> unless changed), <code>PROD=1</code> , how people sign in, <code>PLANT_TZ</code> , and <code>SEAL_KEY_FILE</code> and <code>INTEGRITY_KEY_FILE</code> (keys made as in IN-13, kept outside the folder); save and leave. <pre>nano .env</pre>	—	

#	Do this	You should see	Done
6	Make the empty database. <pre>opencore-mes db reset --yes --empty</pre> Cannot be undone. It makes the database named in <code>.env</code> again: everything in it is lost. Check <code>DATABASE_URL</code> in <code>.env</code> names a new database.	An empty database.	
7	Name the first administrator. <pre>opencore-mes admin <id> "<Full name>" --url https://<server name></pre> Cannot be undone. Once per installation: check the sign-in id and the name first.	The first administrator, a one-time link, setup open (as IN-15).	
8	Start it, from inside the folder. <pre>opencore-mes start</pre>	listening on ...; the sign-in page at the address.	
9	For scripts walled in (Linux): see where the package is, then set <code>SCRIPT_RUNNER_WRAP</code> in <code>.env</code> to <that folder>/@opencore-mes/server/ops/script-runner-sandbox.sh (an absolute path), and install its AppArmor profile beside it as in IN-16. <pre>npm root -g</pre>	/healthz : "network":"none" .	

Caution. The package holds the application and the script runner's walls only. Backups, the audit trail's protection and host hardening are in `ops/` of the source archive of the same version: use them as in IN-11, IN-17 and IN-18, with this folder's database.

3.1 On a laptop

A laptop runs the same OpenCore MES, for trying it, training, a demo or designing offline. It is never a plant's live system: nothing backs its data up, it sleeps when its lid closes, and nobody else should reach it. Prepare the laptop (IN-31 on a Mac, IN-32 on Windows), then IN-33.

IN-31 Prepare a MacBook

Purpose	Node and PostgreSQL on macOS (Apple silicon or Intel), in Terminal.
Done by	Whoever uses the laptop
Before you start	A macOS that Homebrew supports; an administrator's account; Homebrew installed (it prints the line that adds it to your PATH: run it).

#	Do this	You should see	Done
1	Node 24 or later, from the macOS installer at nodejs.org (LTS), or from Homebrew. Then, in a new Terminal window, check it. <pre>node -v</pre>	v24.1... or later (≥24).	
2	Only if Node came from the nodejs.org installer: give npm a global folder of your own. Otherwise installing a package for every user fails with EACCES on /usr/local/lib/node_modules ; never work around it with sudo. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.zshrc && source ~/.zshrc npm config get prefix</pre>	Your ~/.npm-global .	
3	PostgreSQL, started now and with the Mac. <pre>brew install postgresql@17 brew services start postgresql@17</pre>	Successfully started postgresql@17 .	
4	Check that PostgreSQL lets you in. (If psql is not found, the next step.) <pre>psql -d postgres -c 'select current_user'</pre>	Your macOS user name: Homebrew's PostgreSQL lets you in as yourself, over its local socket, as a superuser. So postgres:///<database> needs no password here.	
5	Only if psql was not found: see where Homebrew put PostgreSQL, add the bin folder it names to your PATH as it says, open a new Terminal, and do the step before again. <pre>brew info postgresql@17</pre>	The folder, and the line to add.	

The script runner's operating-system walls are Linux's (bubblewrap): on a Mac scripts run inside Node's own permission model only, and /healthz says so. That is fine on a laptop; never set SCRIPT_ISOLATION=required there.

IN-32 Prepare a Windows laptop

Purpose	OpenCore MES on Windows 10 or 11, inside WSL 2 (Ubuntu running inside Windows): everything then runs on Linux, as on a server.
Done by	Whoever uses the laptop
Before you start	Windows 10 (version 2004 or later) or Windows 11; an administrator's account; virtualisation on in the firmware (it is on most laptops).

#	Do this	You should see	Done
1	In PowerShell, opened as administrator: install Ubuntu in WSL. Restart Windows when asked. <pre>wsl --install -d Ubuntu-24.04</pre>	Ubuntu opens and asks for a Linux user name and password (your own, for this laptop).	
2	From here on, in Ubuntu (Start → Ubuntu): PostgreSQL, and bubblewrap for the script runner's walls. <pre>sudo apt-get update sudo apt-get install -y postgresql bubblewrap</pre>	PostgreSQL 16 installed.	
3	Start PostgreSQL, and let your Linux user in (a superuser, on a laptop only). <pre>sudo service postgresql start sudo -u postgres createuser --superuser "\$USER"</pre>	CREATE ROLE : postgres:///<database> needs no password for you.	
4	Node 24 or later in Ubuntu: from nodejs.org (Download → Linux, with a version manager such as nvm). Then check it. <pre>node -v</pre>	v24.... or later (>=24).	
5	Only if Node came another way (Ubuntu's packages, a tarball in /usr): give npm a global folder of your own. Otherwise installing a package for every user fails with EACCES . With nvm, skip this step. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.bashrc && source ~/.bashrc npm config get prefix</pre>	Your ~/.npm-global .	
6	Each time Windows restarts, in Ubuntu, before starting OpenCore MES: <pre>sudo service postgresql start</pre>	—	

Work inside Ubuntu's own folders (~), not on /mnt/c : it is much faster. Open the address it prints in a browser on Windows: WSL passes localhost through. If it does not open, set HOST=0.0.0.0 in the settings: WSL's network stays inside the laptop unless you forward it, but never leave the sample people's picker on a network others reach. OpenCore MES on Windows outside WSL is not tested.

IN-33 Install and run it on a laptop

Purpose	OpenCore MES running on the laptop: the sample plant to try it (way A), or an empty plant of your own (way B). From npm, or from a source archive: do the steps for one, skip the other's.
Done by	Whoever uses the laptop
Before you start	IN-31 or IN-32 done. In Terminal (Mac) or Ubuntu (Windows).

#	Do this	You should see	Done
1	From npm. On a Mac, only if Node came from the nodejs.org installer (Homebrew's Node and nvm need nothing): give npm a folder of your own, so the install below does not fail with EACCES. Once per Mac. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.zshrc && source ~/.zshrc</pre>	npm config get prefix then says your ~/.npm-global.	
2	From npm. On Linux or Windows (Ubuntu), only if Node came from Ubuntu's packages (nvm needs nothing): the same, for bash. Once per machine. <pre>mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global echo 'export PATH="\$HOME/.npm-global/bin:\$PATH"' >> ~/.bashrc && source ~/.bashrc</pre>	The same.	
3	From npm: install the package, make a plant folder, go into it. <pre>npm install -g @opencore-mes/server opencore-mes init my-plant cd my-plant</pre>	A plant folder with its .env.	
4	From npm, way A (the sample plant): make the database with the sample people (Dana, Iris, Sam...) and plant (lots, machines, work orders). <pre>opencore-mes db reset --yes</pre> Cannot be undone. It makes the database named in .env again: everything in it is lost. On a laptop's own new database only.	It says what it loaded.	
5	From npm, way B (a plant of your own): the empty database, and its first administrator. <pre>opencore-mes db reset --yes --empty opencore-mes admin <id> "<Full name>" --url http://localhost:9090</pre> Cannot be undone. The first line makes the database again (everything in it is lost); the second runs once per installation: check the id and name first.	The first administrator, and a one-time link to set their password; setup open.	
6	From npm: start it. For way B add PROD=1 to .env first, for the real sign-in page. <pre>opencore-mes start</pre>	... listening on http://127.0.0.1:9090 ...	

#	Do this	You should see	Done
7	From a source archive: get the source and its dependencies. <pre>git clone https://github.com/opencore-mes/opencore-mes.git cd opencore-mes npm ci</pre>	The source, with its three dependencies.	
8	From source, way A: the database with the sample plant. <pre>DATABASE_URL=postgres:///opencore_mes node app/mes/db/reset.mjs</pre> Cannot be undone. It makes the database opencore_mes again: everything in it is lost.	It says what it loaded.	
9	From source, way B: the empty database, and its first administrator. <pre>DATABASE_URL=postgres:///opencore_mes node app/mes/db/reset.mjs --empty DATABASE_URL=postgres:///opencore_mes node app/mes/db/admin.mjs <id> "<Full name>" --url http://localhost:9090</pre> Cannot be undone. The first line makes the database again (everything in it is lost); the second runs once per installation: check the id and name first.	The first administrator, and a one-time link to set their password; setup open.	
10	From source, way A: start it, on this laptop only. <pre>HOST=127.0.0.1 DATABASE_URL=postgres:///opencore_mes node app/mes/server.mjs</pre>	... listening on http://127.0.0.1:9090 (development ...)	
11	From source, way B: start it, on this laptop only, with the real sign-in page. <pre>HOST=127.0.0.1 PROD=1 DATABASE_URL=postgres:///opencore_mes node app/mes/server.mjs</pre>	... listening on http://127.0.0.1:9090 (production ...)	
12	Open http://localhost:9090. Way A: pick who you are. Way B: open the password link first. With PROD=1 over plain http://localhost, use Chrome, Edge or Firefox: the sign-in cookie is marked Secure, which they accept on localhost.	Way A: the people to pick from (no passwords: a development instance). Way B: the sign-in page.	
13	To stop it: Ctrl+C in the terminal where it runs. To start again: the start step above.	—	

To remove it, the database, then the folder; and, if nothing else uses them, the package and PostgreSQL (on Windows, unregistering Ubuntu removes the whole Ubuntu, everything in it included):

```
dropdb opencore_mes
```

Cannot be undone. The database and everything in it are deleted.

```
npm uninstall -g @opencore-mes/server
```

```
brew services stop postgresql@17  
brew uninstall postgresql@17
```

On Windows, in PowerShell:

```
wsl --unregister Ubuntu-24.04
```

Cannot be undone. The whole Ubuntu is deleted: OpenCore MES, its database, and every other file in it.

Caution. A development instance (without `PROD=1`) lets anyone who reaches it sign in as anyone. Keep it on 127.0.0.1 on a laptop: the plant folder's `.env` says so already; from source, `HOST=127.0.0.1` as in the start steps. It warns at start when it is reachable from other machines.

4. Upgrades

A release changes the application; the plant's designs live in its database and change only through change requests, so an upgrade leaves them as they are. The database upgrades itself: at each start, every new or changed migration runs, in order, under one lock (several instances starting together run each once). Migrations go forward only.

IN-40 Upgrade a server

Purpose	A new release, with its database changes, and a way back.
Done by	IT
Before you start	- The release notes read: anything to do by hand is said there. - A time agreed with the plant: the service restarts.

#	Do this	You should see	Done
1	Make a backup now, and note its file name: it is the way back (IN-42). <pre>systemctl start opencore-mes-backup.service ls -lt /var/backups/opencore-mes head -3</pre>	The newest opencore_mes- <stamp>.dump.enc .	
2	Install the new release beside the current one: IN-12 steps 1 to 3, with its own folder name (not step 4 yet).	—	
3	Optionally, the test pipeline in the new release (IN-21 step 2, in its folder).	all ... stages passed .	
4	Switch to it, and restart. <pre>ln -sfn /srv/opencore-mes/releases/<new build> /srv/opencore-mes/current systemctl restart opencore-mes</pre> Cannot be undone. Once it has started, the new release's migrations have changed the database, and they go forward only: the previous release cannot run on it again. The way back is the backup of step 1 (IN-42).	—	
5	See what migrated, and ask it how it is. <pre>journalctl -u opencore-mes -n 50 -o cat curl -fsS http://127.0.0.1:3000/healthz</pre>	database: applied ... for each migration (the event db.migrated names them); ok true; the new build.	
6	If the trail is protected and the release notes say a migration changes it: run that migration and the protection script as postgres (IN-17).	—	
7	The first upgrade to a release with the data integrity review: an integrity reviewer opens Design → Data integrity and signs Take the baseline .	From then on a change made around the platform is found.	

#	Do this	You should see	Done
8	Keep the last few releases in <code>/srv/opencore-mes/releases</code> ; remove older ones.	—	

IN-41 Upgrade a plant folder (npm)

Purpose	A new version of the npm package.
Done by	IT

#	Do this	You should see	Done
1	Make a backup first (IN-18 step 5, or <code>backup.sh</code> by hand), and stop the server (Ctrl+C, or its service).	—	
2	Install the new version. <pre>npm install -g @opencore-mes/server@<version> opencore-mes version</pre>	The new version.	
3	From inside the plant folder: bring the database up to date (or let the start do it). <pre>opencore-mes db migrate</pre> Cannot be undone. The database's changes go forward only: the previous version cannot run on it again. The way back is the backup of step 1.	What migrated, or database: up to date.	
4	Start it. <pre>opencore-mes start</pre>	listening on ...; /healthz answers.	

IN-42 Going back after a bad upgrade, or restoring a backup

Purpose	The previous release running again, on the data as it was before the upgrade.
Done by	IT, with the plant's agreement

#	Do this	You should see	Done
1	If the new release never started (its migrations did not run): point <code>current</code> back at the previous release, and restart. Then stop here. <pre>ln -sf /srv/opencore-mes/releases/<previous build> /srv/opencore-mes/current systemctl restart opencore-mes</pre>	The previous build in <code>/healthz</code> .	
2	Otherwise its migrations have changed the database, and the previous release cannot be pointed at it. Stop the service. <pre>systemctl stop opencore-mes</pre>	—	

#	Do this	You should see	Done
3	Give the service's user the backup taken before the upgrade, its checksum and the key, for the restore only. <pre>install -d -m 0700 -o opencore /var/lib/opencore-mes/restore install -m 0400 -o opencore /var/backups/opencore-mes/<file>.dump.enc /var/backups/opencore-mes/<file>.dump.enc.sha256 /etc/opencore-mes/backup.key /var/lib/opencore-mes/restore/</pre>	—	
4	Restore it into a new database, as the service's user (so the service owns it, as it owned the old one); then remove the copies. <pre>cd /srv/opencore-mes/current sudo -u opencore env PGHOST=/var/run/postgresql BACKUP_KEY=/var/lib/opencore-mes/restore/backup.key TARGET_URL=postgres:///opencore_mes_<date> ops/backup/restore.sh /var/lib/opencore-mes/restore/<file>.dump.enc rm -rf /var/lib/opencore-mes/restore</pre> Cannot be undone. The last line removes only the copies made in step 3; the backups in /var/backups stay.	Checksum checked, restored, and its audit chain verified (verify-audit.mjs --full). It refuses a target that exists.	
5	Open the settings file and point DATABASE_URL at postgres:///opencore_mes_<date>; save and leave. <pre>nano /etc/opencore-mes/opencore-mes.env</pre>	—	
6	Point current at the previous release, and start. <pre>ln -sf /srv/opencore-mes/releases/<previous build> /srv/opencore-mes/current systemctl start opencore-mes</pre> Cannot be undone. Everything recorded after the backup was taken is not in the restored database. Keep the upgraded database aside: it is the only place those records remain.	The previous build, on the data of the backup.	
7	Tell the plant what is lost: everything recorded between the backup and the stop. Run the protection script on the restored database (IN-17, with its name), and keep the upgraded database aside until it is no longer needed.	—	

Caution. There is no rollback command: going back is this procedure. A backup before every upgrade is what makes it possible.

5. Suites

A suite is an npm package (`@opencore-suites/<name>`) that adds designs, blocks, steps or equipment adapters, from the suites registry, with your licence token. The core works the same with any suite removed: what a suite leaves stays inert and labelled, never deleted, and works again once it is reinstalled; every version an installation ran is kept. (The suites store is not open yet.)

IN-50 Install, update, remove a suite (a plant folder)

Purpose	A suite in an installation, and every version it ran kept.
Done by	IT
Before you start	- A licence token for the suites registry (or the suite's <code>.tgz</code> for a plant without internet access). - In the plant folder (on a server, IN-51 first).

#	Do this	You should see	Done
1	Sign in to the registry; paste the token when asked (it is not shown). <pre>opencore-mes suite login</pre>	The token is kept in <code>.local/suites/.npmrc</code> (mode 0600), for this installation only.	
2	Install the suite. <pre>opencore-mes suite install <name></pre>	<code><name> <version></code> installed in <code>suites/<name></code> .	
3	Without internet access, instead: install it from its file. <pre>opencore-mes suite install <name> --from <file>.tgz</pre>	The same.	
4	Restart OpenCore MES.	The suite's migrations run; it is listed in the designer, and its designs can be started from it.	
5	Later, to see, update, remove, or go back to a kept version (restart after each): <pre>opencore-mes suite list opencore-mes suite update <name> opencore-mes suite remove <name> opencore-mes suite use <name>@<version></pre>	A replaced or removed version is kept in <code>suites/.versions/</code> , never deleted; use puts one back.	

IN-51 Suites on a server

Purpose	The suite commands on a server installed as part 2, with the suites kept outside the releases.
Done by	IT

#	Do this	You should see	Done
1	Open the settings file and add <code>SUITES_DIR=/var/lib/opencore-mes/suites</code>; save and leave. <pre>nano /etc/opencore-mes/opencore-mes.env</pre>	—	
2	Open a shell as the service's user, in its data folder. <pre>sudo -u opencore bash</pre>	The prompt changes to <code>opencore@... .</code>	
3	In that shell: load the settings, and go to the data folder. <pre>set -a; . /etc/opencore-mes/opencore-mes.env; set +a cd /var/lib/opencore-mes</pre>	—	
4	In that shell, run the suite commands of IN-50, with the command's full path in front instead of <code>opencore-mes</code>, as here: <pre>node /srv/opencore-mes/current/app/mes/cli.mjs suite login node /srv/opencore-mes/current/app/mes/cli.mjs suite install <name></pre>	As in IN-50.	
5	Leave the shell, and restart the service. <pre>exit systemctl restart opencore-mes</pre>	—	

6. Reference

6.1 Settings

Setting	Default	What
PROD=1	off	Production: no sign-in picker, Secure cookies, a snapshot of the pages.
HOST , PORT	0.0.0.0, 9090	Where it listens: 127.0.0.1 behind Caddy.
DATABASE_URL , PGHOST	development's own: always set it	The database (a plant's: postgres:///opencore_mes); PGHOST the socket's folder.
PLANT_TZ	the server's	The time zone of schedules that name none.
EVENT_LOG_DIR	.local/events	Where the event log file is written.
TRUST_PROXY	unset	How many proxies are in front (1 behind Caddy), so the client's address is the real one.
SEAL_KEY_FILE , INTEGRITY_KEY_FILE	none	The two keys (IN-13), or SEAL_KEY , INTEGRITY_KEY themselves.
INTEGRITY_FULL_HOURS	168	How often the integrity scan reads every record.
OIDC_ISSUER , OIDC_CLIENT_ID , OIDC_CLIENT_SECRET , OIDC_CLAIM , OIDC_LABEL	—	Single sign-on. The redirect address is https://<server>/login/sso/callback ; the claim is the sign-in id (preferred_username unless set).
LDAP_URL , LDAP_USER_DN , LDAP_CA_FILE , LDAP_LABEL	—	The directory: ldaps://host:636 , and whom an id binds as ({user}@plant.local).
PASSWORDS=0	on	No passwords of OpenCore MES's own.
MFA	optional	required or off : an authenticator's code after a password.
SIGN_WITH_PASSWORD=0	on	Signatures ask the signer's password (or a fresh single sign-on).
PASSWORD_MAX_DAYS , PASSWORD_HISTORY	90, 5	Password expiry and reuse.
SESSION_IDLE_MINUTES	30	Sign-out after that long with nothing done (12 hours at most).
PASSWORD_LINK_DAYS	3	How long a one-time password link lasts unless its maker says (1 to 14 days).
SCRIPT_RUNNER_WRAP , SCRIPT_ISOLATION=required	none	The script runner's walls (IN-16).
MES_CONNECTION_HOSTS	any host	The hosts services may reach (comma separated).
ALERT_WEBHOOK_URL , ALERT_MIN_SEVERITY	none, error	Alerts (IN-19).
SETUP=1	off	Setup at the first start of an installation not made empty (the first administrator of an empty one opens it already).

Setting	Default	What
REPLICA_URL, BUS=1, INSTANCE	—	Reads from a replica; several instances on one database (app/mes/README.md, Scaling).
AI_PROVIDER, ANTHROPIC_API_KEY, AI_MODEL	none	The copilot.
SUITES_DIR, SUITES_REGISTRY	suites/, the store's	Where suites are, and where they come from.

The full list, with every option: the header of `app/mes/server.mjs` and `app/mes/README.md`.

6.2 Files and folders

Path	What
/srv/opencore-mes/releases/<build>	Each release; <code>/srv/opencore-mes/current</code> points at the one that runs.
/etc/opencore-mes/opencore-mes.env	The settings (root:opencore, 0640).
/etc/opencore-mes/seal.key, integrity.key, backup.key	The keys (copies off the server).
/etc/opencore-mes/backup.env	What is backed up, where, and where it is copied.
/var/lib/opencore-mes/events	The event log file (also copied into the database).
/var/backups/opencore-mes	The encrypted backups (postgres, 0700).
/etc/systemd/system/opencore-mes.service; opencore-mes-backup.*; opencore-mes-restore-test.*	The service and the backup timers.

6.3 Console commands

Command (on a server: <code>node app/mes/...</code> with the settings loaded)	What
<code>db/reset.mjs --empty · opencore-mes db reset --yes --empty</code>	An empty database. Everything in the database is lost; refused on a protected one.
<code>db/admin.mjs <id> "<name>" · opencore-mes admin</code>	The first administrator of an empty installation.
<code>db/password.mjs <id> --url <site> · opencore-mes password</code>	A one-time link for a person to set their password.
<code>db/token.mjs <user> "<name>" [--days 1..365] · opencore-mes token</code>	A token for an integration user.
<code>db/migrate.mjs · opencore-mes db migrate</code>	The database brought up to date (the server does it at start).
<code>db/verify-audit.mjs [--full]</code>	The audit chain checked by hand.
<code>db/seal-secrets.mjs</code>	Second-factor secrets kept before a seal key was set, sealed now.
<code>db/recover-designer.mjs <designer> [--reviewer <user>] "<why>"</code>	Design roles given back when nobody can change the system any more; refused otherwise, audited.

6.4 When something refuses

You see	Why, and what to do
The service stops at start: the script runner has no walls	<code>SCRIPT_ISOLATION=required</code> without bubblewrap or its AppArmor profile: install them (IN-16), or remove the setting while you do.
Signing in says nobody has been added yet	The database is empty: name the first administrator (IN-15).
<code>npm install -g: EACCES</code> , permission denied on <code>/usr/local/lib/node_modules</code>	npm's global folder belongs to root (Node from the nodejs.org installer on a Mac, or Ubuntu's packages): give npm one of your own (IN-31 step 2, IN-32 step 5), not <code>sudo</code> .
<code>opencore-mes</code> : unknown command <code>admin</code> , or <code>--empty</code> loads the sample people	The npm package is 0.1.0, from before them: try the sample plant with it, or install from a source archive (IN-33) until the next package.
<code>admin.mjs</code> : People & departments has ... already	The first administrator is named once; add people through a change, or recover with <code>recover-designer.mjs</code> if nobody can change the system.
<code>reset.mjs</code> : a protected database is never reset	Its audit trail is protected (IN-17): it is a plant's. Restore a backup into a new database instead (IN-42).
<code>/healthz</code> answers 503	The database is not answering, or the service is stopping: <code>journalctl -u opencore-mes</code> .
A migration fails at start on a protected database	It changes the audit trail: run it as postgres, then <code>protect-audit.sql</code> (IN-17).
<code>SETUP=1</code> ignored	Changes were already reviewed and approved there: setup is opened again by a change governance approves.

6.5 Revision history

Revision	Date	What changed
A	2026-10-06	First issue.
B	2026-10-06	Laptops: a MacBook and a Windows laptop (WSL 2), the sample plant or a plant of one's own (IN-31 to IN-33).
D	2026-10-07	MacBook: npm's global folder of one's own (EACCES with the nodejs.org installer); two more refusals explained.
D	2026-10-07	IN-20: everyone's first password with setup codes (a file for printed slips), and moving to the plant's directory; the sign-in id's label, hint and domains.